

RAD Control

Sistema de Gestão de Solicitações Acadêmicas

Relatório Técnico

Nome completo:

JOÃO GABRIEL NEVES FERRER

Disciplina: RAD - Desenvolvimento Rápido de Aplicações

Professor: Abraão Henrique

Data:

12/06/2026 - 14/06/2026

1. Descrição do Problema Resolvido

Antes do desenvolvimento do RAD CONTROL, a coordenação controlava as solicitações dos alunos de forma manual, utilizando papéis físicos. Essa abordagem gerava desorganização, pois os documentos se espalhavam por diferentes locais da sala, podendo se misturar com outros arquivos, resultando em estresse e perda de informações importantes.

Com o objetivo de solucionar esse problema, foi desenvolvido o RAD Control, um sistema desktop que permite registrar e gerenciar solicitações dos alunos de forma simples e intuitiva. O sistema armazena todas as informações em um banco de dados PostgreSQL, garantindo que nenhuma solicitação seja perdida.

Por meio da interface gráfica desenvolvida com Tkinter, o usuário consegue cadastrar novas solicitações, acompanhar o status de cada uma, pesquisar por nome do aluno, prioridade ou situação, além de atualizar e excluir registros quando necessário. Dessa forma, o RAD Control transforma um processo antes caótico em algo organizado, ágil e confiável.

2. Bibliotecas e Tecnologias Utilizadas

Python

Foi a linguagem principal para o desenvolvimento do RAD Control. Com ela foi criado todas as funções que faziam cada botão a interface funcionar de uma maneira, validação dos formulários, integração com o banco de dados e funcionalidades do sistema. Sua simplicidade e organização permitiu um ótimo desenvolvimento de uma aplicação desktop funcional e de fácil manutenção

Tkinter

Biblioteca padrão do python para criação de interfaces gráficas amigáveis para o gerenciamento das solicitações acadêmicas. Neste projeto foi usado para construir a janela principal da aplicação, além de botões, rótulos, campos de entrada, caixas de texto e eventos de interação com o usuário.

ttk (themed Tkinter)

Ttk é o conjunto de componentes visuais modernos que complementa o tkinter. No projeto os componentes usados foram: Combobox, para seleção de opções, e Treeview, para exibição dos registros em formato de tabela.

PostgreSQL

É um sistema gerenciador de banco de dados relacional responsável pelo armazenamento das informações do sistema. Foi usado para guardar os registros de cada aluno e suas solicitações acadêmicas de forma segura e estruturada, permitindo cadastro, consulta, atualização e exclusão de dados.

psycopg2

É o adaptador que permite a comunicação entre o Python e o PostgreSQL. No projeto foi usado para estabelecer uma conexão com o banco de dados, executar comandos SQL, realizar consultas, inserir novos registros, atualizar informações e excluir dados quando precisar.

Faker

Biblioteca que gera dados fictícios para testes. No projeto ela foi empregada no arquivo seed.py para criar pelo menos 20 registros simulados de alunos e solicitações acadêmicas.

Datetime

Biblioteca nativa do python para manipular datas. No projeto foi utilizada no arquivo seed.py para gerar prazos aleatórios de forma dinâmica, calculando datas futuras a partir da data atual.

3. Modelo da Tabela do Banco de Dados

A tabela principal do sistema se chama `solicitacoes` e foi criada no banco `rad_db` com os seguintes campos:

Campos da tabela `solicitacoes`

id — Chave primária, gerada automaticamente (SERIAL)

aluno_nome — Nome do aluno (VARCHAR 120, obrigatório)

matricula — Matrícula do aluno (VARCHAR 30, obrigatório)

tipo — Tipo da solicitação: Dúvida, Entrega, Correção, Orientação, Revisão ou Outro

prioridade — Nível de prioridade: Baixa, Média ou Alta

status — Situação atual: Aberto, Em andamento, Concluído ou Cancelado

descricao — Texto livre com detalhes da solicitação

data_abertura — Data e hora do cadastro (preenchida automaticamente)

prazo — Data limite para atendimento (opcional)

```
rad_db=# \d solicitacoes
```

Coluna		Tabela "public.solicitacoes"			Padrão
	Tipo	Ordenação	Pode ser nulo		
id	integer		not null		nextval('solicitacoes_id_seq'::regclass)
aluno_nome	character varying(120)		not null		
matricula	character varying(30)		not null		
tipo	character varying(40)		not null		
prioridade	character varying(20)		not null		
status	character varying(25)		not null		
descricao	text		not null		
data_abertura	timestamp without time zone				CURRENT_TIMESTAMP
prazo	date				

Índices:

"solicitacoes_pkey" PRIMARY KEY, btree (id)

4. Funcionalidades Implementadas

F1 — Cadastro de Solicitações

O sistema permite o cadastro de novas solicitações acadêmicas por meio de um formulário gráfico desenvolvido em Tkinter. O usuário deve informar o nome do aluno, matrícula, tipo de solicitação, prioridade, status, descrição e prazo. Antes de salvar os dados, o sistema verifica se os campos obrigatórios (nome, matrícula, tipo e descrição) foram preenchidos.

F2 — Listagem em Grade (Treeview)

Os registros cadastrados são amostrados em uma tabela utilizando o componente `ttk.Treeview`. A grade apresenta informações como ID, nome do aluno, matrícula, tipo de solicitação, prioridade, status, descrição, data de abertura e prazo. Para facilitar a visualização, as linhas recebem cores diferentes de acordo com a prioridade da solicitação: verde para baixa, amarelo para média e vermelho para alta.

F3 — Pesquisa e Filtro

O sistema possui um campo de pesquisa que permite localizar registros armazenados no banco de dados. A busca é realizada através da integração com o banco de dados e os resultados encontrados são exibidos diretamente na Treeview. Após a pesquisa, o usuário também pode utilizar botão “recarregar” para visualizar novamente todos os registros cadastrados.

F4 — Atualização de Registros

Funciona através da seleção de um registro na Treeview. Ao clicar em uma linha na tabela, os dados são carregados automaticamente nos campos do formulário. O usuário pode mudar as informações desejadas e utilizar o botão “Atualizar” para salvar essas modificações no banco de dados.

F5 — Exclusão com Confirmação

A exclusão é realizada por meio de seleção prévia de uma solicitação na tabela. Antes da remoção definitiva, o sistema exibe uma caixa de diálogo de confirmação utilizando o método `askyesno()` da biblioteca `messagebox`. Dessa forma o usuário evita excluir registros por engano. Após a confirmação o registro é removido do banco de dados e a tabela é atualizada automaticamente.

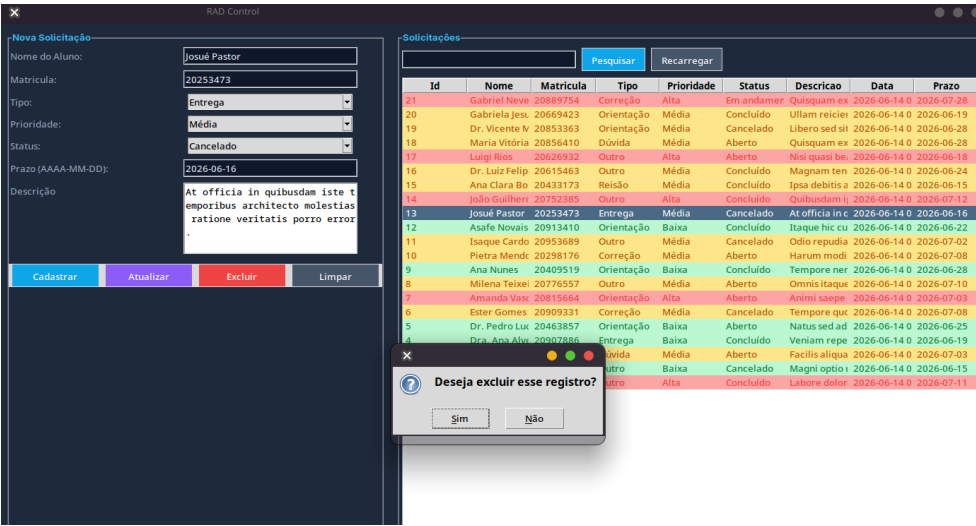
F6 — Geração de Dados Fictícios (seed.py)

No sistema, com a biblioteca `Faker` configurada no padrão brasileiro (`pt_BR`), gera automaticamente nomes de alunos, matrículas, descrições, tipos de solicitações, prioridades, status e prazos aleatórios. Foram colocados 20 registros fictícios para facilitar os testes do funcionamento do CRUD.

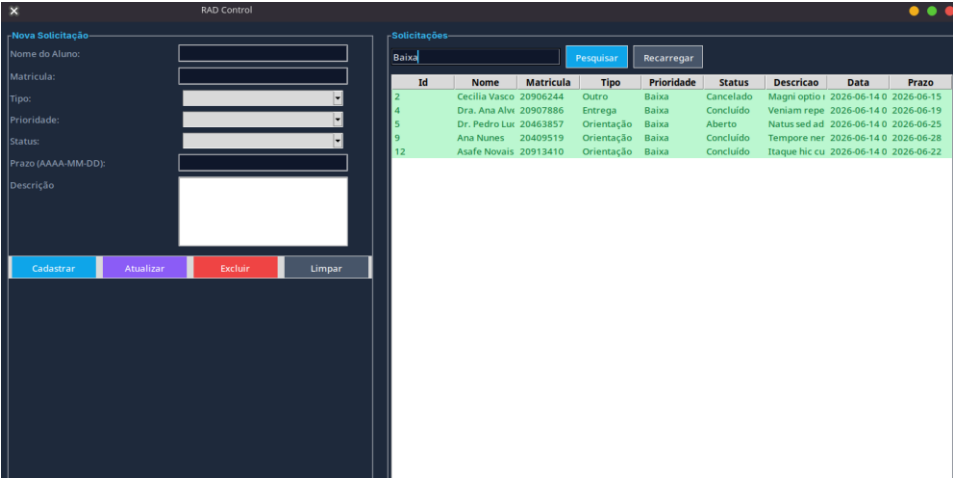
5. Prints da Aplicação Funcionando

Cole abaixo os 6 prints obrigatórios com uma legenda para cada um.

Print 1 — Tela inicial com registros listados



Print 5 — Pesquisa funcionando



Print 6 — Banco de dados com registros

```
~/Documentos/Sistema-de-gest-o-de-usuarios---RAD-Control main*  
venv > sudo -u postgres psql -d rad_db -c "SELECT id, aluno_nome, status FROM solicitacoes;"  
id |      aluno_nome      | status  
-----  
1 | Melissa Gomes        | Concluido  
2 | Cecilia Vasconcelos  | Cancelado  
3 | Sr. Caleb Leão       | Aberto  
4 | Dra. Ana Alves       | Concluido  
5 | Dr. Pedro Lucas Câmara | Aberto  
6 | Ester Gomes          | Cancelado  
7 | Amanda Vasconcelos   | Aberto  
8 | Milena Teixeira      | Aberto  
9 | Ana Nunes            | Concluido  
10 | Pietra Mendonça      | Aberto  
11 | Isaque Cardoso        | Cancelado  
12 | Asafe Novais          | Concluido  
14 | João Guilherme das Neves | Concluido  
15 | Ana Clara Borges      | Concluido  
16 | Dr. Luiz Felipe Cardoso | Concluido  
18 | Maria Vitória Macedo  | Aberto  
19 | Dr. Vicente Moura     | Cancelado  
17 | Luigi Rios            | Aberto  
21 | Gabriel Neves        | Em andamento  
20 | Gabriela Jesus        | Concluido  
(20 linhas)
```


6. Dificuldades Encontradas e Como Foram Resolvidas

Dificuldade 1 - Conexão entre Python e PostgreSQL

Uma das maiores dificuldades foi trabalhar com PostgreSQL no ambiente Linux. Diferente do Windows, onde muitas configurações podem ser realizadas por interfaces gráficas, no Linux tive que utilizar comando no terminal para iniciar e gerenciar o serviço do banco de dados. Em alguns momentos tiveram erros na aplicação porque o PostgreSQL não estava em execução, por muitas vezes eu esquecer de ligar ele por conta que o comando era meio grande. Para resolver esse problema foram criados aliases para facilitar o uso dos comandos mais utilizados.

Dificuldade 2 - Integração das Funções da Interface com o CRUD

Durante o desenvolvimento do sistema, minha dificuldade foi integrar de forma certa os eventos da interface gráfica do Tkinter com os métodos responsáveis pelas operações CRUD no arquivo database.py. Em diversos momentos ocorreram erros devido à passagem incorreta de argumentos para as funções ou pela utilização inadequada de métodos. Resolvi esse problema revisando a estrutura do código e testando cada funcionalidade individualmente.

Dificuldade 3 - Personalização da Interface Gráfica

Outra dificuldade encontrada, que era um ponto opcional, mas eu estava com tempo livre para fazer, foi a estilização da interface. De início eu sabia a base de deixar a janela mais estilizada com cores nos botões e no background, porém, eu tive problemas na configuração de cores da treeview de acordo com o nível de prioridade das solicitações. Em alguns testes as cores eram aplicadas de forma errada ou não apareciam como esperado. Depois de ter reajustado as tags, foi possível organizar a interface.

7. Conclusão

Durante o desenvolvimento do projeto, aumentei meus conhecimentos sobre a criação de interfaces gráficas com Python e integração dessas interfaces com banco de dados. Antes da atividade, eu já possuía uma noção básica de Tkinter e sabia como realizar conexões simples com um banco de dados utilizando Python. No entanto, ao desenvolver o sistema na prática, percebi que ainda tem muita coisa que não sabia sobre essas duas bibliotecas.

Além disso, aprendi funções, métodos e formas de organizar o código que tornaram o desenvolvimento mais eficiente. Também ganhei mais experiência trabalhando com PostgreSQL em ambiente Linux, o que me ajudou a entender melhor a administração do banco de dados e sua integração com aplicações. Ao final do projeto, me sinto confiante para desenvolver novos sistemas em Python utilizando Tkinter e banco de dados, além de ter uma base mais sólida para aprender e utilizar outras bibliotecas no futuro, como: Django, Flask, Streamlit etc.